

Corrigé : Évaluation Développement Frontend Moderne

1. QCM (Réponses)

- 1. Web Component
- 2. `useState`
- 3. `getStaticProps`
- 4. Atome
- 5. Il optimise le rendu en limitant les manipulations du vrai DOM

2. Vrai/Faux (Réponses et justifications)

- 1. **Faux.** Le DOM peut être modifié dynamiquement avec JavaScript, pas uniquement en HTML.
- 2. **Vrai.** Un composant React peut contenir plusieurs appels à `useState` pour gérer différents morceaux d'état.
- 3. **Vrai.** Les Web Components peuvent être utilisés dans n'importe quelle application JS, y compris React, bien qu'il faille parfois adapter les props/events.
- 4. **Faux.** `useEffect` s'exécute *après* le rendu du composant, pas avant.
- 5. **Faux.** Dans Next.js, les pages sont générées automatiquement à partir du dossier `/pages`, il n'y a pas de fichier de configuration à maintenir pour cela.

3. Questions courtes (Réponses attendues)

- 1. Le Shadow DOM permet d'encapsuler le style et le comportement d'un composant Web. Il évite les conflits de styles avec le reste de la page.
- 2. Les `props` sont des données passées par un composant parent, tandis que le `state` est géré en interne dans un composant. Le `state` peut évoluer, pas les `props`.
- 3. En Atomic Design, je crée d'abord des composants très simples (atoms), que je combine en composants plus complexes (molecules, organisms), pour finir avec des pages complètes. Cela améliore la lisibilité et la réutilisabilité.
- 4. `getStaticProps` est utilisé pour générer une page au moment du build, tandis que `getServerSideProps` exécute la récupération de données à chaque requête côté serveur.
- 5. Le Virtual DOM permet de comparer deux versions d'une interface et de ne mettre à jour que les éléments modifiés dans le DOM réel. Cela rend les mises à jour plus rapides.

4. Étude de cas (Réponses détaillées attendues)

- 1. **Technologie recommandée :** Next.js
Justification : permet de combiner performance, SEO, et génération de pages dynamiques, tout en utilisant React.
- 2. **Structure avec Atomic Design :**
 - `/components/atoms/Button.js`
 - `/components/molecules/ActivityCard.js`
 - `/components/organisms/SearchForm.js`
 - `/pages/index.js`, `/pages/activities/[id].js`
- 3. **Récupération des données :**
Utilisation de `getStaticProps` ou `getServerSideProps` dans les fichiers de page Next.js. Sinon, `fetch` dans un `useEffect` si côté client uniquement.
- 4. **Optimisations possibles :**
Lazy loading des images, préchargement des routes, mise en cache des données, affichage des skeletons pour réduire l'impression de latence.

5. Rédaction technique (Réponses détaillées attendues)

Sujet 1 (Carte de produit)

- Créer un composant React ou Web Component prenant en entrée des props/attributs (`title`, `image`, `price`).
- Utiliser une structure HTML claire et styliser avec CSS modules ou encapsulé via Shadow DOM.
- Le composant est réutilisable sur différentes pages produits ou dans une liste.

Sujet 2 (Page dynamique dans Next.js)

- Créer un fichier `[id].js` dans le dossier `/pages/produits/`.
- Utiliser `getStaticPaths` pour générer les routes, et `getStaticProps` pour charger les données à partir d'un appel API.
- Afficher les données du produit en fonction de l'ID.

Sujet 3 (Notation en étoiles)

- Créer un composant `StarRating` avec un état interne ou un prop `rating`.
- Afficher dynamiquement des icônes d'étoiles remplies ou vides.
- Ajouter des listeners au clic pour modifier la note. Gérer l'accessibilité avec `aria-label` et la navigation clavier.

Total de l'évaluation : 60 points.